

UDRIVE MANUAL TÉCNICO

Sistema de gestión y simulación de viajes

Alex Ricardo Castañeda Rodríguez
Carné 202300476 - Sección B
IPC1 - Práctica 2

1. Descripción general

UDrive es una aplicación de escritorio desarrollada en Java 21 y Swing. Administra rutas cargadas desde CSV, asigna tres pilotos a una flota fija de nueve vehículos y ejecuta hasta tres recorridos concurrentes. Cada viaje controla distancia, combustible, retorno, fechas e historial.

Objetivos técnicos

- Separar interfaz, lógica de aplicación, dominio y persistencia.
- Representar rutas, vehículos, pilotos y viajes mediante objetos serializables.
- Ejecutar cada recorrido en un hilo independiente sin bloquear Swing.
- Conservar el estado en un archivo binario mediante ObjectOutputStream.
- Producir un JAR autónomo y verificable con Maven.

Tecnologías

Componente	Uso
Java 21	Lenguaje, serialización, concurrencia y API de archivos
Swing + FlatLaf	Interfaz gráfica y estilo visual
Maven Wrapper	Compilación, pruebas y generación del JAR
JUnit 5	Pruebas de dominio, CSV, viajes y persistencia

2. Arquitectura de la solución

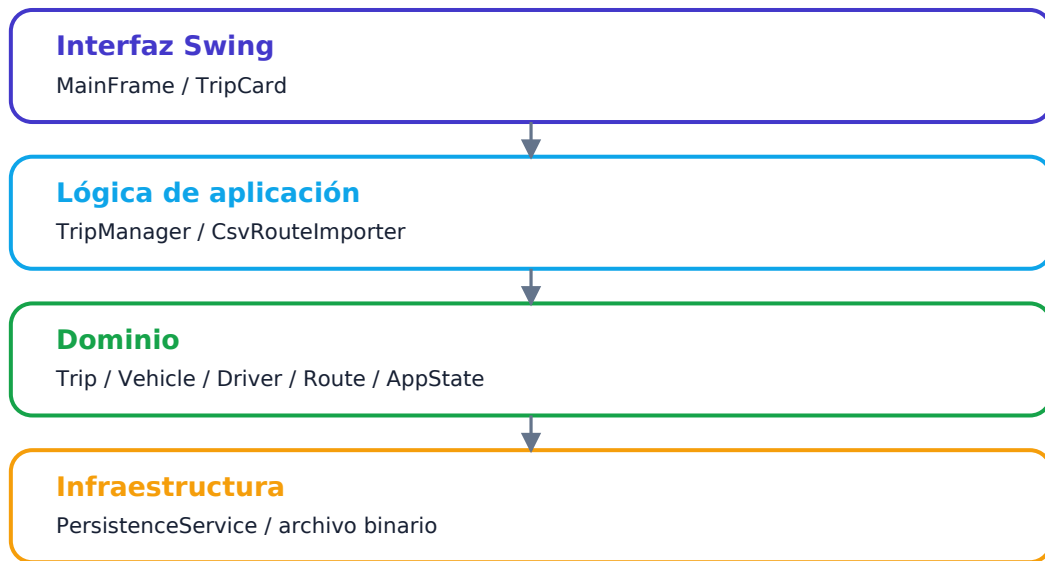


Figura 1. Capas y dependencias principales.

MainFrame se limita a mostrar datos y enviar acciones. TripManager valida y coordina los casos de uso. Las entidades mantienen el estado del negocio. PersistenceService escribe AppState primero en un archivo temporal y luego lo reemplaza de forma atómica para reducir el riesgo de corrupción.

Estructura del proyecto

Ruta	Responsabilidad
src/main	Código fuente y punto de entrada
src/vehicles	Recursos visuales de la flota
test/main	Pruebas automatizadas
examples/rutas.csv	Archivo de rutas de ejemplo
data/udrive-state.bin	Estado binario creado en ejecución
target/UDrive.jar	Ejecutable autónomo generado

3. Diagrama de clases

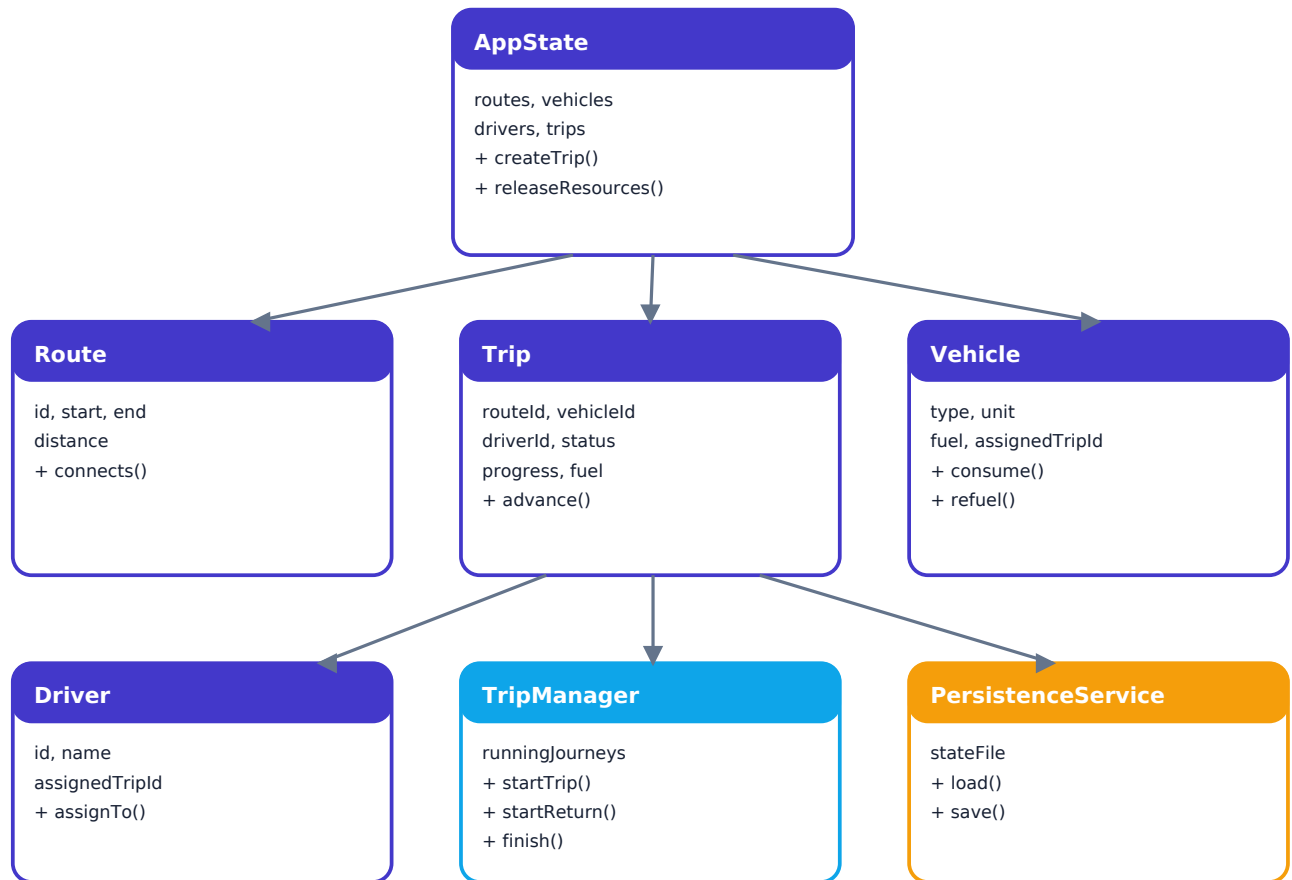


Figura 2. Clases principales y asociaciones.

Responsabilidades

Clase	Responsabilidad principal
AppState	Agrega rutas, flota, pilotos, viajes y contadores
Trip	Mantiene estado, progreso, fechas, distancia y consumo
Vehicle	Controla asignación, tanque, consumo y recarga
Driver	Controla la disponibilidad de cada piloto
TripManager	Coordina generación, inicio, retorno y finalización
Journey	Hilo que avanza el recorrido a velocidad uniforme
PersistenceService	Serializa y restaura AppState

4. Flujo de carga de rutas

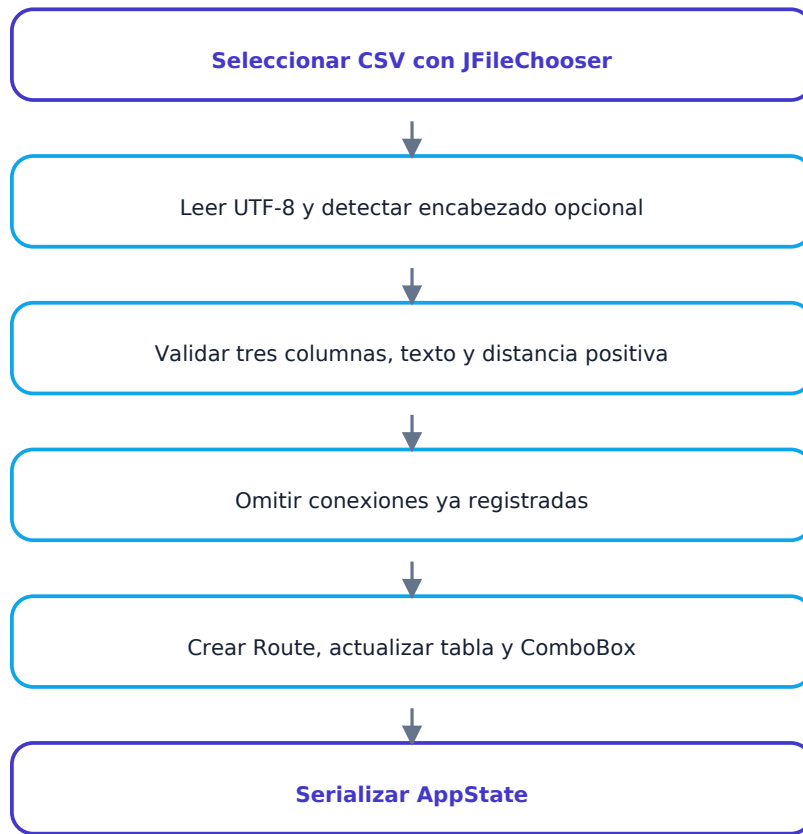


Figura 3. Flujo del método de importación.

Una ruta se considera bidireccional: Route.connects compara ambos sentidos. El identificador se asigna desde nextRouteId y se conserva al reiniciar la aplicación.

5. Ciclo de vida de un viaje



Figura 4. Estados persistentes de Trip.

- PREPARED reserva un piloto y un vehículo, pero aún no inicia el hilo.
- OUTBOUND y RETURNING avanzan un kilómetro por ciclo de simulación.
- OUT_OF_FUEL conserva el progreso y bloquea únicamente el hilo de ese viaje.
- WAITING_RETURN permite iniciar el regreso o finalizar en el destino.
- COMPLETED registra la hora final y libera piloto y vehículo.

6. Flujo concurrente, combustible y retorno

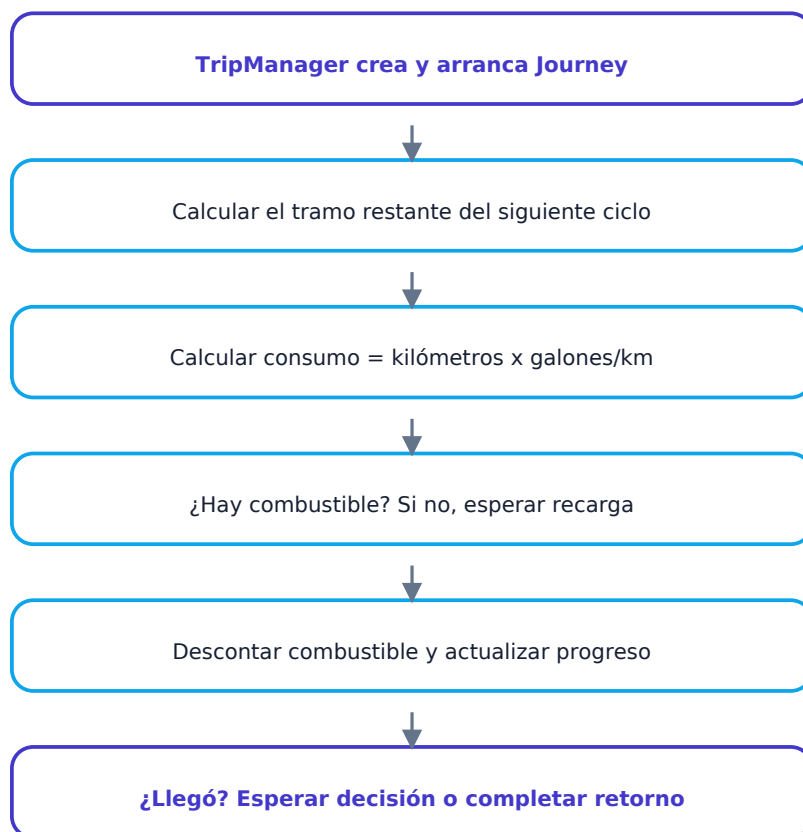


Figura 5. Algoritmo principal de Journey.run().

Parámetros de la flota

Tipo	Unidades	Consumo (gal/km)	Tanque (gal)
Motocicleta	3	0.10	6
Vehículo estándar	3	0.30	10
Vehículo premium	3	0.45	12

7. Persistencia binaria

AppState y sus entidades implementan Serializable. PersistenceService guarda el grafo completo de objetos en data/udrive-state.bin. El archivo no es CSV ni JSON; contiene un flujo binario de objetos Java.

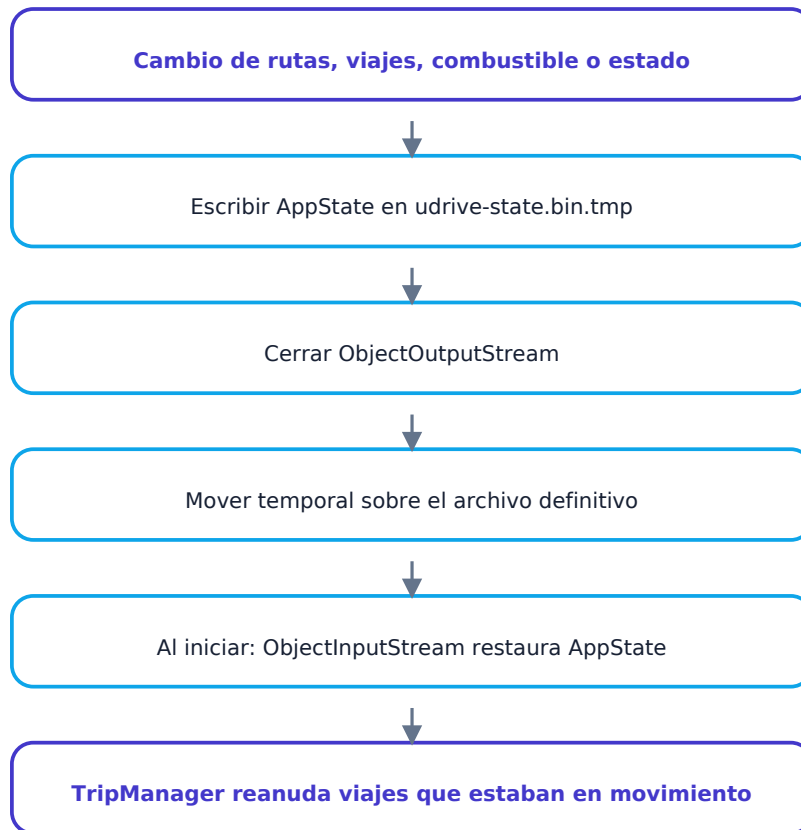


Figura 6. Guardado seguro y recuperación.

Al cerrar la ventana se detienen los hilos, se guarda su progreso actual y se conservan los estados OUTBOUND, RETURNING o OUT_OF_FUEL. En el siguiente inicio se crean hilos nuevos desde el kilómetro persistido.

8. Construcción, ejecución y pruebas

Compilar y verificar

Linux/macOS: ./mvnw clean verify

Windows: mvnw.cmd clean verify

Ejecutar

java -jar target/UDrive.jar

Cobertura de pruebas

Grupo	Validación
AppStateTest	Flota, asignación, liberación y serialización
CsvRouteImporterTest	CSV válido, columnas y distancias
JourneyTest	Avance, consumo, falta de combustible y recarga
TripManagerTest	Ida, retorno, historial y liberación de recursos

Decisiones de seguridad y estabilidad

- La interfaz se crea en el Event Dispatch Thread de Swing.
- Cada Journey es daemon y actualiza Swing mediante invokeLater.
- Los métodos que alteran recursos compartidos están sincronizados.
- El archivo temporal evita reemplazar el último estado válido antes de completar la escritura.
- El JAR sombreado incorpora FlatLaf y los recursos visuales.